

エスカレシミュレーター要件定義書

アプリケーション概要

概要

項目	内容
システム名	エスカレシミュレーター
システム概要	社外の問い合わせを、 緊急度 ・ 契約プラン ・ 待ち時間 から「今どれを先にやるか」の点数にまとめ、手が空いた担当者がその1件を引き取る仕組み。待ち順の変化は画面にすぐ反映する
利用対象	申請者(customer)、サポート担当者(operator)、管理者(admin)
開発目的	「緊急度」と「顧客の契約プラン」を、担当者が迷わない1つの対応順にする。複数の担当者が同時に動いている様子も見えるようにする
ユースケース	ITヘルプデスク / カスタマーサポートの対応順シミュレーター

解決したい課題

用語

- ・ 担当者は、問い合わせに返信するサポートの人です。複数の担当者は、その人が同時に2人以上ログインしている状態です。二人がほぼ同時に「次のタスクを引き取る」を押すと、仕組みがないと同じ問い合わせが二人に渡ります。
- ・ VIPは、特別なログイン権限ではありません。契約プランが Enterprise（有料の上位）の顧客のことです。
- ・ 安いプランは Free（無償）です。その中間が Pro です。
- ・ 全停止は、サービス全体が使えない障害です。質問や軽い不具合より、必ず先に対応します。
- ・ 対応期限は、問い合わせを作ってから、初めて担当者が付くまでの目標時間です。Free は 120 分、Pro は 60 分、Enterprise は 15 分です。
- ・ 点数は、緊急度・プラン・期限切れ・待ち時間を1つの数字にしたものです。大きいほど先に対応します。

プラン

プラン	意味	点数	初めて担当が付くまでの目標
Free	無償	1	120分
Pro	有償	2	60分
Enterprise	重要顧客（VIP）	3	15分

先に届いた順だけで対応すると、全停止の障害や重要顧客が後回しになる。待ち時間を無視すると、安いプランの人がいつまでも待たされる。複数の担当者が同時に「次のタスクを引き取る」を押すと、同じ件が二人に渡ることがある。本システムは、点数による並び、一人にしか渡さない引き取り、対応期限の監視、画面の即時更新でこれを防ぐ。

課題	内容
到着順だと重大障害が埋もれる	緊急度が高いほど先にする。全停止は質問より必ず先
VIPと一般の扱いが人によって違う	同じ緊急度の中では、契約プランが高い顧客を先にする
安いプランが長く待たされ続ける	対応期限を過ぎた件は、まだ時間内の高いプランより先にする
同じチケットが二人に渡る	「いちばん先の1件を取り出す」と「その人の担当にする」を組み合わせ、同時押しでも一人にしか渡さない
緊急度を変えても並びがすぐ変わらない	点数をやり直して待ち一覧を更新し、接続中の画面へ知らせる
対応期限を過ぎても気づかない	プランごとの目標時間を監視し、過ぎたら赤くする
誰が空いているか分からない	担当者の待機中 / 作業中 / 離席と、今持っているチケットをすぐ見えるようにする

実装する機能

機能	内容
ユーザー認証	会員登録、ログイン、ログアウト。役割（申請者 / 担当者 / 管理者）を付ける
顧客一覧	契約プラン（Free / Pro / Enterprise）と、初回対応の対応期限（分）を持つ。申請者は必ずどこかの顧客に属する
チケット起票	件名・詳細・緊急度・種類を入れて登録する。
優先度の自動計算	緊急度・契約プラン・対応期限切れ・待ち時間から点数を出す。計算の式は「点数のルール」だけを書く場所に置く（保存先の都合を混ぜない）
待ち一覧	点数が高い順に並ぶ。管理者と担当者は、順位の变化をすぐ見る
次のタスクを引き取る	担当者がボタンを押すと、今いちばん先の対応待ちが自動でその人の担当になる。同時押しは早い者勝ちで、同じ件は一人だけ
緊急度の変更	待ち中または対応中に緊急度を上げると、待ち一覧の位置がすぐ変わる
対応期限の警告	プランごとの「何分以内に初めて担当が付くか」を過ぎたら赤く表示し、担当者与管理者へ知らせる
対応完了	終了コメントと、引き取りから完了までの時間を残す。担当者は待機中に戻る
ダッシュボード	今待っている件数、平均の対応時間、対応期限切れ件数をすぐ表示する
デモ	管理者がダミーの問い合わせを連続で流し、同時に引き取る様子を再現する
セキュリティ	JWT、連続操作の回数制限、CSRF、XSS対策
API	REST API、WebSocket API

技術スタック

フロントエンド	言語は TypeScript。フレームワークは React。開発とビルドは Vite。見た目は Tailwind CSS
画面遷移	画面から画面への移動は React Router
バックエンド	言語は Go。API のフレームワークは Echo
ORM	データベースへの読み書きは GORM
データの本体	チケットの正しい状態の保存は PostgreSQL
待ち順の高速参照	点数順の並びと、連続操作の回数制限は Redis
認証	ログイン中であることの確認は JWT
画面の即時更新	開いている画面への知らせは Gorilla WebSocket
API	通常の操作は REST API。待ち順の変化の通知は WebSocket
インフラ	まとめて起動する環境は Docker
テスト	サーバーのテストは Go testing。画面のテストは Vitest
CI/CD	ビルド、テスト、デプロイの自動化は GitHub Actions

ユーザー

ログイン後の役割は、アカウント作成時または管理者が付ける。1アカウントは1役割とする。

ゲスト

機能	実装内容	理由	完了条件
会員登録	メール、表示名、パスワードを入れてアカウントを作る。最初の役割は申請者	サービスを使えるようにするため	正しい入力でアカウントができる
ログイン	メールとパスワードで本人確認する	登録した本人として使うため	正しい情報でログインできる

申請者(customer)

社外の顧客に属する人。チケットを登録し、自分の件の進み具合だけを見る。

機能	実装内容	なぜ必要？	完了条件
ログアウト	再ログイン用の印を無効にし、認証Cookieを消す	安全に使い終わるため	同じ印では再ログインできない
顧客の扱い	社外の申請者は所属顧客が固定。デモでは管理者が付け替える	契約プランと対応期限を決めるため	登録した時点でプランが決まっている
チケット起票	件名・詳細・緊急度・種類を入れて登録する	問い合わせを待ち一覧に載せるため	対応待ちで作られ、点数順の並びに入る
自分のチケット一覧	自分が出した件の状態・担当者・おおよその待ちを見る	進み具合を知るため	他人の件や、全体の待ち一覧は見えない
チケット詳細	本文、完了後のコメント、対応期限を過ぎているかを見る	回答を受け取るため	その件の内容が見える

顧客そのものではなく、その顧客のアカウントを持つ人です。

所属

申請者は、必ず1つの社外顧客に属します。Free、Pro、Enterprise のどれかは、所属している顧客で決まります。起票画面でプランは選びません。担当者と管理者は顧客に属しません。所属がまだ無い申請者は、チケットを作れません。所属を付けるのは管理者です。本番相当の申請者は所属が固定で、デモでは管理者が付け替えることができます。

できること

- ・ ログアウトする。再ログイン用の印を無効にし、認証 Cookie を消す。
- ・ チケットを起票する。入力するのは件名、詳細、緊急度、種類だけ。
- ・ 自分が出したチケットの一覧を見る。状態、担当者、おおよその待ちが分かる。並びは更新が新しい順。
- ・ 自分のチケットの詳細を見る。本文、完了後のコメント、対応期限を過ぎているかが分かる。

ホームは `/tickets` です。使える画面は `/tickets`、`/tickets/new`、`/tickets/:ticketId` です。

通知

画面の即時更新では、自分のチケットだけが届きます。作られた、引き取られた、緊急度が変わった、完了した、待ちに戻った、対応期限を過ぎた、という通知です。

できないこと

- ・ 他の申請者のチケットを見る。無い件も、他人の件も、どちらも「見つからない」と返す。
- ・ 全体の待ち一覧、現場の数字、他人の本文を見る。
- ・ 次を引き取る、緊急度を変える、対応メモを書く、対応を完了にする。
- ・ 顧客や利用者を登録・変更する。

サポート担当者(operator)

複数人が同時にログインできる。手が空いたときに、今いちばん先の1件を引き取る。

機能	実装内容	なぜ必要？	完了条件
稼働状態	待機中 / 作業中 / 離席を切り替える。「次のタスクを引き取る」が成功すると自動で作業中になる	空いている人だけが次のタスクを取るため	状態が保存され、他の人の画面にも出る
待ち一覧を見る	対応待ちを点数の高い順に並べて見る	今の対応順を把握するため	順位がすぐ更新される
次のタスクを引き取る	今いちばん先の対応待ちを、自分の担当にする	人が好きな件だけ拾わないようにするため	1件だけ自分に付き、他の担当者には同じ件が渡らない
担当中の対応	引き取った件の詳細を見て、メモを残せる	対応内容を残すため	自分に付いた件だけ操作できる
緊急度を上げる	自分が対応中の件の緊急度を上げる	調べてみたら全停止相当だった、というときに順を上げるため	緊急度が上がる。まだ対応待ちなら待ち一覧の位置がすぐ変わる。対応中なら履歴に残る

機能	実装内容	なぜ必要？	完了条件
対応完了	終了コメントを入れて完了にする	次の件へ進むため	完了になり、自分は待機中に戻る

サポート担当者（operator）は、問い合わせに返信する人です。同時に2人以上ログインできます。顧客には属しません。手の空いた人が、待ちの先頭の1件を引き取ります。好きな件だけを選ぶことはできません。

稼働

状態は3つです。待機中は次のタスクを引き取れます。作業中は対応中の件を1件持っています。離席は次のタスクを引き取りません。作業中へ自分で切り替えることはできません。作業中になるのは、引き取りが成功したときだけです。自分で切り替えられるのは、待機中と離席です。

同時に持てる対応中は1人あたり1件です。

できること

- 待ち一覧を見る。対応待ちが、点数の高い順に並びます。同じ点数なら、先に作られた件が先です。対応期限を過ぎた行は赤くします。
- 次のタスクを引き取る。押せるのは待機中で、対応中の件を持っていないときだけです。待ちの先頭を1件取り出し、その人の担当にします。二人同時に押しても、同じ件は一人にしか渡りません。
- 自分の対応中の件の詳細を見て、対応メモを残す。
- 自分の対応中の件の緊急度を上げる。調べてみたら全停止相当だったときに使います。下げることはできません。
- 終了コメントを入れて完了にする。完了後、自分は待機中に戻ります。
- 離席にする。対応中の件があれば、担当を外して待ち一覧に戻します。対応期限の起算は、作った時刻のままです。

ホームは `/agent/queue` です。使える画面は `/agent/queue` と `/agent/tickets/:ticketId` です。

通知

待ち順の変化は、担当者全員に届きます。作られた、引き取られた、緊急度が変わった、完了した、待ちに戻った、対応期限を過ぎた、担当者の稼働が変わった、という知らせです。つながった直後に、今の待ち一覧と担当者の空きが届きます。詳細の本文は、待ち一覧の知らせには入れません。

画面との接続がすべて切れて5分戻らなければ、離席と同じく担当を外し、待ち一覧に戻します。

できないこと

- 新しい問い合わせを1件登録する。
- 作業中や離席のときに、次のタスクを引き取る。
- 緊急度を下げる。完了した件の緊急度を変える。
- 他人が対応中の件を操作する。見られるのは、待ち一覧にある対応待ち、自分が対応中の件、自分が完了にした件です。
- ダッシュボードの数字、顧客の登録、利用者の役割変更、デモの開始。現場全体の数字は管理者だけです。

管理者(admin)

待ち一覧全体、対応期限、担当者の空きを見る。問い合わせの登録はしない（必要なら申請者アカウントを別に使う）。

機能	実装内容	なぜ必要？	完了条件
ダッシュボード	待ち件数、平均の対応時間、対応期限切れ件数、担当者の空きを表示する	現場全体を見るため	後述の定義どおりの数字が、すぐ更新される
全件の監視	すべての状態のチケットを検索する	個別の件を追うため	条件に合う件が一覧になる
緊急度の変更	待ち中・対応中を問わず緊急度を上げられる。下げるのは管理者だけ	付け間違いや事態の悪化に対応するため	点数と待ち一覧が更新される
顧客の管理	顧客名・プラン・対応期限（分）を登録・更新する	プランと約束を維持するため	変更後に新規登録した件から新しい値が使われる
利用者の管理	役割、所属顧客、アカウントの状態を変える	デモと運用で人を用意するため	対象の権限が変わる

機能	実装内容	なぜ必要？	完了条件
デモ	何件・何秒おきにダミーを自動登録するか指定する	並びの変化と、同時に引き取る様子を見せるため	指定どおり件が作られ、待ち一覧とダッシュボードが動く

管理者（admin）は、現場全体を見る人です。待ち一覧、対応期限、担当者の空きを確認します。顧客には属しません。問い合わせの登録はしません。必要なら、別の申請者アカウントを使います。

できること

- ダッシュボードを見る。待ち件数、対応中件数、平均の対応時間、平均の初回応答、今の対応期限切れ、直近24時間の対応期限切れ、担当者の空き（待機中・作業中・離席の人数）です。
- 全件を監視する。状態、緊急度、顧客、担当者で絞り込めます。
- 待ち中と対応中の緊急度を上げる。下げることもできます。下げるのは、付け間違いを直すときです。完了した件は変えられません。
- 対応メモを残す。対応完了にもできます。担当を外して、待ち一覧へ戻すこともできます。
- 顧客を登録・更新する。名前、プラン（Free / Pro / Enterprise）、初めて担当が付くまでの目標時間（分）です。
- 利用者を管理する。役割、申請者の所属顧客、アカウントの状態を変えます。
- 任意の担当者を離席にできる。
- デモを実行する。作る件数（1〜50）、間隔（1〜10秒）、出方（均等 / 障害多め）を指定し、通常の起票と同じ道でダミーを流します。

ホームは `/admin/dashboard` です。使える画面

は `/admin/dashboard`、`/admin/tickets`、`/admin/tickets/:ticketId`、`/admin/customers`、`/admin/users`、`/admin/demo` です。

通知

待ち順の変化は、管理者全員に届きます。つながった直後に、今の待ち一覧とダッシュボードの数字が届きます。その後は、関連する知らせと、30秒ごとの数字のまとめで更新します。デモの進みは、実行した管理者にだけ届きます。

できないこと

- チケットを起票する。デモの自動登録は、管理者の画面から通常の起票処理を呼びますが、管理者自身が問い合わせを書くのとは別です。
- 次のタスクを引き取る。
- 完了した件の緊急度を変える。
- 申請者や担当者の画面に、現場全体の数字を出す。

業務ルール

顧客の区分と契約プラン

区分	説明	プランの点数 (plan)	初回対応の約束 (担当が付くまでの目標)
社外 Free	無償プラン	1	120分
社外 Pro	有償プラン	2	60分
社外 Enterprise	VIP / 大口	3	15分

- 申請者アカウントは必ず1つの顧客に属する。
- 起票画面でプランを手入力しない。所属顧客から取る。
- デモ用に、申請者が顧客を切り替えられるサンプルデータを置いてよい。本番相当の申請者は所属が固定。

緊急度

値	名称	例
4	システム全停止	サービス全体が使えない
3	一部機能不可	主要機能が使えない
2	不具合報告	回避できる不具合
1	質問・要望	使い方、改善のお願い

チケットの状態

状態	意味
対応待ち	まだ担当が付いていない。待ち一覧に載る
対応中	担当者が決まっている
完了	対応が終わった。待ち一覧から外す

一度完了にした件を元に戻す機能は、最初の版では作らない。間違えて完了にした場合は、管理者が新しい件として出し直す。

優先度の点数

点数の計算は「ルールを書く場所」だけが持つ。Redis や PostgreSQL の都合を知らない。チケットを作ったとき、緊急度を変えたとき、定期的なやり直しのときに、この計算を呼んで順番メモリを更新する。

```

点数 =
  緊急度 × 1_000_000
+ 対応期限切れなら 10_000 (まだなら 0)
+ プランの点数 × 100
+ 待ち分数 (99分で打ち止め)

```

材料	定義
緊急度	1〜4
対応期限切れ	まだ担当が付いておらず、待ち分数が対応期限以上なら 1。担当が付いたあとは待ち一覧にいないので、この計算の対象外
プランの点数	1〜3
待ち分数	作ってから今までの分数（秒は切り捨て）。99を超えたら99として扱う

先にやる順番の意味

1. 緊急度がいちばん強い。緊急度が1違うと、待ち時間やプランでは追い越せない。
2. 緊急度が同じなら、対応期限を過ぎた件を、まだ時間内の高いプランより先にする。「Free が対応期限を過ぎているのに、Enterprise の新しい件を先にする」ことはしない。
3. 緊急度が同じで、どちらも時間内（またはどちらも切れ）なら、プランが高い方を先にする。
4. それでも同じなら、待っている時間が長い方を先にする。
5. 待ち時間で加点できるのは最大99なので、時間内ではプランの差（100）を追い越せない。プランの差を超えるのは、対応期限切れの加点（10000）だけ。

やらないこと

- 画面表示のときだけ点数を出し、順番メモリには到着順だけ置くやり方。一覧に出ている順と、実際に引き渡す1件がずれるのでやらない。
- 待ち時間だけで Enterprise の新しい件を追い越すやり方。プランごとに決めた約束と合わないのでやらない。オーバー判定は、その顧客の対応期限（分）を使う。

点数をいつやり直すか

タイミング	処理
チケット作成	点数を出し、PostgreSQL に保存し、順番メモリにも載せる

タイミング	処理
緊急度の変更	緊急度を変えたあと点数をやり直し、データベースと順番メモリを更新する
30秒ごとの処理	対応待ちの点数をやり直して順番メモリを更新する。待ち時間と、対応期限切れが変わるため
引き取り / 完了	順番メモリからその件を外す。点数のやり直しは不要

30秒は、いちばん短い対応期限（15分）より十分短い。画面への通知が多すぎない間隔でもある。

担当者の空きと割り当て

規則	内容
「次のタスクを引き取る」ができる状態	待機中だけ。作業中・離席はボタンを拒否する
引き取り成功	担当者は作業中。チケットは対応中。誰が・いつ引き取ったかを記録する
完了成功	チケットは完了。完了時刻と終了コメントを残す。担当者は待機中に戻る
自分で離席にする	対応中の件は対応待ちに戻し、待ち一覧に載せ直す。担当者で引き取り時刻は空にする。対応期限の起算は「作った時刻」のまま（一度離席したことで遅れを帳消しにしない）
画面との接続が切れたとき	切れてから5分戻らなければ、離席と同じく担当を外して待ち一覧に戻る
同時に持てる件数	1人あたり対応中は最大1件

緊急度を変えられる人

操作	申請者	担当者	管理者
緊急度を上げる	できない	自分が対応中の件だけ	対応待ち・対応中のすべて
緊急度を下げる	できない	できない	対応待ち・対応中のすべて
完了した件の緊急度変更	できない	できない	できない

- ・ 対応期限の起算は、いつも「作った時刻」。緊急度を変えても時計は振り直さない（初回対応の遅れを隠さない）。
- ・ 下げるのは付け間違いの修正用。誰が何から何へ変えたかを記録に残す。

初回対応の対応期限

項目	定義
何を測るか	作ってから、初めて担当が付くまで。この時間が対応期限を超えたらオーバー
完了までの時間	オーバー判定には使わない。平均の対応時間の数字に使う
離席で担当が外れたあと	引き取り時刻は空になる。再び待っている間も、起算は作った時刻のまま
画面	オーバーした件は赤くする。担当者と管理者へ、その件について1回知らせる。担当が付いたら赤は消す（待ちではなくなる）
監視のやり方	30秒ごとの点数やり直しの中で、「まだ知らせていないオーバー」を見つける。Redis の期限切れだけに頼ると、待ち一覧の色とずれるので、それ単独では使わない

ダッシュボードの数字

数字	定義
待ち件数	対応待ちの件数
対応中件数	対応中の件数

数字	定義
平均の対応時間	直近24時間に完了した件について、「最後に引き取った時刻」から「完了した時刻」までの平均。途中で離席して別の人が引き取った場合は、最後の引き取り時刻を使う
平均の初回応答	直近24時間に初めて担当が付いた件について、「作った時刻」から「引き取った時刻」までの平均
今の対応期限切れ	対応待ちで、作ってから対応期限以上経っている件数
24時間の対応期限切れ	直近24時間に、担当が付いた時点ですでにオーバーしていた件数
担当者の空き	待機中 / 作業中 / 離席の人数

申請者の画面には、現場全体の数字は出さない。

デモ

項目	内容
誰が実行するか	管理者だけ
入力	作る件数（1～50）、間隔秒（1～10）、緊急度とプランの出方（均等 / 障害多め）
処理	サンプル顧客を無作為に選び、ダミー件名で普通の起票と同じ道を通す
停止	管理者が止める。生成中でも、すでにある件の引き取り・完了は普通どおり
目的	待ち順の変化、赤い警告、複数担当者の早い者勝ちを画面で見せる

実装手順

順	実装内容	なぜ必要？	完了条件
会員登録	名前、メール、パスワードでアカウントを作る。最初は申請者	サービスを使えるようにするため	正しい入力でアカウントができる
ログイン	メールとパスワードで本人確認する	本人として各機能を使うため	正しい情報でログインできる
ログアウト	再ログイン用の印を無効にし、認証Cookieを消す	共用のパソコンでも安全に終わるため	ログアウト後、同じ印では新しい認証を出せない
連続操作の制限	会員登録、ログイン、再発行、起票、引き取りなど、連続すると影響が大きい操作に回数上限を付ける	パスワードの総当たりや大量リクエストを防ぐため	上限以内は処理し、超えたら 429 を返す
CSRF対策	再ログイン用の印をCookieで送り、再発行とログアウトでは CSRF 用Cookieとヘッダーを照合する	他のサイトからCookieを悪用されないため	正しい CSRF 用の値が無いと処理しない
暫定トップ	ログイン後に表示名、役割、ログアウトを出す	認証の直後に動くか確認するため	ログイン後にトップが出る。未ログインならログイン画面へ
画面遷移	公開画面、ログイン必須、役割ごとの画面を分ける	権限のない画面を開かせないため	役割に合った画面だけ出る
ログイン状態の共有	ユーザー情報、役割、確認中かどうかをアプリ全体で持つ	画面ごとに食い違わないため	再読み込みしてもログインが残る
顧客	顧客、プラン、対応期限（分）を登録する。最初から各プランのサンプルを入れる	点数と対応期限の材料にするため	顧客が一覧でき、プランが取れる
申請者と顧客	申請者を1顧客へ付ける	起票時にプランを決めるため	申請者のプランが決まる
点数	計算の関数と、後述の判定表のテストを書く	ビジネスのルールを、保存先の都合から離すため	判定表をすべて満たす
チケット起票	件名・詳細・緊急度・種類を保存し、待ち一覧に載せる	問い合わせを受け付けるため	チケットができ、順番メモリに入る
待ち一覧	対応待ちを点数の高い順に返す	順位を確認するため	一覧が点数順である
画面の即時更新	ログイン後、役割に応じた通知を受け取る	並びの変化をすぐ見るため	つながった直後に、今の待ち一覧が届く

順	実装内容	なぜ必要？	完了条件
次のタスクを引き取る	いちばん先の1件を取り出し、データベース上も「対応待ちのときだけ」その人の担当にする	同時押しでも一人にしか渡さないため	同時に押しても同じ件が二人に付かない
対応完了	コメントと時刻を残し、担当者を空ける	次の件へ進むため	完了になり、待ち一覧から消える
緊急度の変更	緊急度を変えて点数をやり直す	事態の悪化を順にすぐ反映するため	待ち一覧の位置と画面が更新される
対応期限の監視	30秒ごとに点数のやり直しと、オーバーの検知を行う	時間が経つと順と警告が変わるため	オーバーした件が赤くなり、知らせが届く
ダッシュボード	数字を集めて表示する	管理者が状況を把握するため	定義どおりの数字が出る
デモ	ダミー起票を自動で流す	シミュレーターとして見せるため	作った件が普通の待ち一覧に載る

画面遷移

項目	内容
実装内容	React Router で画面の移動を管理する
公開画面	<code>/login</code> 、 <code>/register</code>
申請者	<code>/tickets</code> 、 <code>/tickets/new</code> 、 <code>/tickets/:ticketId</code>
担当者	<code>/agent/queue</code> 、 <code>/agent/tickets/:ticketId</code>
管理者	<code>/admin/dashboard</code> 、 <code>/admin/tickets</code> 、 <code>/admin/tickets/:ticketId</code> 、 <code>/admin/customers</code> 、 <code>/admin/users</code> 、 <code>/admin/demo</code>
未ログイン	ログインが必要なURLを開くとログイン画面へ
役割	自分の役割以外の画面は拒否し、APIも拒否する
ログイン済み	ログイン中にログイン画面や登録画面を開くと、役割ごとのホームへ
完了条件	URLごとに正しい画面が出て、ログインと役割で入れる場所が分かれる

申請者のホームは `/tickets`、担当者は `/agent/queue`、管理者は `/admin/dashboard`。

ログイン状態の共有

項目	内容
実装内容	ログイン状態を React 全体で共有する
持つ情報	ログイン中の人、役割、所属顧客、ログイン済みか、確認中か
できること	ログイン、ログアウト、今のユーザー取得、状態の更新
起動時	今ログイン中の人を取得する
再読み込み	もう一度ログイン状態を確認する
ログアウト	情報を消し、ログイン画面へ
完了条件	どの画面でも同じログイン状態で、再読み込み後も残る

顧客の登録

項目	内容
実装内容	契約プランと対応期限（分）を持つ顧客を登録・更新する
登録内容	顧客名、プラン、対応期限（分）
制約	プランは free / pro / enterprise
利用権限	管理者
完了条件	顧客が保存され、起票時のプラン取得に使える

申請者と顧客の結びつき

項目	内容
実装内容	申請者を1つの顧客へ付ける
処理内容	ユーザーに顧客IDを保存する
制約	担当者与管理者は顧客に属さない。申請者は必須
利用権限	管理者
完了条件	申請者の起票で、所属顧客のプランと対応期限が使われる

点数の計算

項目	内容
実装内容	緊急度・プラン・対応期限切れ・待ち時間から整数の点数を返す
置く場所	ルール専用。Redis・データベース・HTTP を知らない
入力	緊急度、プラン、作った時刻、今の時刻、対応期限（分）
出力	点数（大きいほど先）
利用権限	システム内部。画面は結果だけ見る
完了条件	次の判定表を自動テストで満たす

点数の判定表（完了条件）

ケース	A	B	先に対応する
緊急度が違う	緊急度4 / Free / 待ち0分	緊急度3 / Enterprise / 待ち99分	A
対応期限切れ vs 高いプランの新規	緊急度2 / Free / 待ち120分（約束120分でオーバー）	緊急度2 / Enterprise / 待ち0分（約束15分でまだ時間内）	A
同じ緊急度・どちらも時間内	緊急度3 / Enterprise / 待ち1分	緊急度3 / Free / 待ち10分	A
同じ緊急度・どちらもオーバー	緊急度2 / Pro / 待ち200分	緊急度2 / Free / 待ち200分	A（プランが高い方）
条件が同じで待ちだけ違う	緊急度1 / Pro / 待ち50分	緊急度1 / Pro / 待ち10分	A

点数が同じときは、**先に作られたチケットを先にする**。作った時刻も同じなら、IDが小さい方を先にする。待ち一覧を返すときは、この順で並べて返す（順番メモリ上の点数が同じでも、画面の順はここで決める）。

順番メモリに載せる点数は整数、載せる値はチケットのIDとする。

チケット起票

項目	内容
実装内容	新しい問い合わせを作る
入力	タイトル、詳細、緊急度（1～4）、種類（障害 / 不具合 / 質問 / 要望 / その他）
自動で行うこと	申請者の所属顧客からプランと対応期限を取る。点数を出す。対応待ちで保存し、順番メモリに載せる。担当者与管理者へ「新しい件」を知らせる
制約	申請者だけが起票できる。デモの自動起票は、管理者権限で同じ登録処理を呼ぶ
利用権限	申請者。デモは管理者
完了条件	チケットができ、待ち一覧に載り、担当者・管理者へ作成の知らせが届く

チケット一覧（申請者）

項目	内容
実装内容	自分が出した件を一覧する
出す項目	タイトル、緊急度、状態、作成日時、対応期限までの残りまたはオーバー、担当者名（付いたあと）
並び	更新が新しい順
制約	他の申請者の件は取れない
利用権限	申請者
完了条件	自分の件だけが一覧になる

待ち一覧（担当者・管理者）

項目	内容
実装内容	対応待ちを、先にやる順に表示する
出す項目	順位、タイトル、緊急度、プラン、待ち時間、点数、対応期限までの残りまたはオーバー、顧客名
並び	点数が高い順。同じ点なら先に作られた順
色	対応期限切れは赤
利用権限	担当者、管理者
完了条件	一覧が点数のルールどおりで、30秒ごとのやり直しと、起票・引き取りなどの知らせで順位が更新される

チケット詳細

項目	内容
実装内容	内容と対応の記録を表示する
表示	タイトル、詳細、緊急度、種類、顧客、プラン、状態、担当者、各時刻、完了コメント、点数
誰が見られるか	申請者は自分の件だけ。担当者は待ち一覧にある対応待ち、自分が対応中の件、自分が完了にした件。管理者はすべて
完了条件	見られる権限のある件の内容を確認できる

次の対応

項目	内容
実装内容	今いちばん先にやるべき対応待ちを、ボタンを押した担当者へ渡す
事前条件	役割が担当者、状態が待機中、対応中の件を持っていない
処理の流れ	1. 順番メモリから、点数がいちばん高いチケットを 1件だけ取り出し、同時に並びから消す。この取り出しは途中で他の担当者の操作が割り込まない（二人が同時でも、同じIDは一人にしか渡らない） 2. データベースで「まだ対応待ちなら」対応中にし、担当者を引き取り時刻を書く 3. 1件更新できたら、担当者を作業中にし、「引き取った」ことを画面へ知らせる 4. 更新できなかった（すでに完了など）ときは、次に点が高い件で同じことを繰り返す。並びへ戻すのは、データベースの書き込み自体が失敗したときだけ
失敗したとき	順番メモリから取り出したあとでデータベースが失敗したら、同じ点数でもう一度並びへ戻す。担当者は待機中のまま

項目	内容
待ちが0件	「待ちチケットがありません」と返す
利用権限	担当者
完了条件	複数の担当者が同時に押しても、同じチケットは一人にしか付かない。渡るのは、その瞬間にいちばん先の1件

対応完了

項目	内容
実装内容	担当中の件を完了にし、コメントと時刻を残す
入力	終了コメント（必須、1～2000文字）
処理	完了にする。 対応時間は「引き取った時刻」から「完了した時刻」。 担当者を待機中に戻す。 「完了した」ことを知らせる
制約	その件の担当者本人、または管理者。対応待ちのままでは完了にできない
利用権限	担当者（自分の件）、管理者
完了条件	待ち一覧に出ず、対応時間とコメントが残る

緊急度の変更

項目	内容
実装内容	チケットの緊急度を変える
入力	新しい緊急度、理由（必須）
処理	緊急度を更新する。 対応待ちなら点数をやり直して順番メモリを更新する。 「緊急度が変わった」ことを知らせる。 記録に残す
制約	上げ下げできる人は「緊急度を変えられる人」の表どおり。 完了した件は変えられない
利用権限	担当者、管理者
完了条件	新しい緊急度が保存され、対応待ちなら待ち一覧の位置がすぐ変わる

稼働状態の更新

項目	内容
実装内容	担当者の待機中 / 作業中 / 離席を更新する
制約	作業中への手動変更は不可（引き取り成功のときだけ作業中）。 待機中と離席は自分で切り替えられる。 対応中の件がある状態で離席にすると、担当を外して待ち一覧に戻る
利用権限	担当者本人。管理者は任意の担当者を離席にできる
完了条件	状態が画面に出る。離席時は担当していた件が待ち一覧に戻る

対応期限の監視

項目	内容
実装内容	対応待ちのまま、初めて担当が付くまでの時間が約束を超えたことを見つける
実行	30秒ごと。作った直後にすでにオーバーすることは、通常ない
処理	まだ知らせていないオーバーに印を付け、担当者と管理者へ知らせる
利用目的	管理者の数字と、待ち一覧の赤い表示
完了条件	オーバーしてから30秒以内に、赤い表示と知らせが届く

ダッシュボード

項目	内容
実装内容	待ち件数、対応中件数、平均の対応時間、平均の初回応答、対応期限切れ件数、担当者の空きを表示する
更新	最初は通常の取得、その後は関連する知らせと30秒ごとのまとめ
利用権限	管理者。担当者画面には自分の状態と待ち一覧だけ（現場全体の数字は管理者）
完了条件	「ダッシュボードの数字」の定義どおり集計される

デモ

項目	内容
実装内容	指定した件数・間隔でダミーの問い合わせを作る
処理	通常の起票と同じ処理を呼び、画面への知らせも同じ
利用権限	管理者
完了条件	作った件が待ち一覧に現れ、引き取れる

画面の即時更新（WebSocket）

接続

項目	内容
実装内容	ログイン後にサーバーと常時つなぐ
認証	つなぐときに Access Token を送り、サーバーで確かめる
誰に何を送るか	役割ごとに、受け取ってよい知らせだけを送る
失敗	未ログイン、期限切れ、権限不足はつながない
切れたとき	一定間隔でつなぎ直し、成功したら今の待ち一覧をもう一度取る
完了条件	ログイン中の人だけがつながり、役割に合った知らせだけを受ける

知らせの種類と相手

知らせ	申請者	担当者	管理者	内容
チケットが作られた	自分が出した件だけ	全員	全員	新しい件。待ち一覧に追加
チケットが引き取られた	自分の件が引き取られたとき	全員	全員	誰がどの件を引き取ったか

知らせ	申請者	担当者	管理者	内容
緊急度が変わった	自分の件の緊急度が変わったとき	全員	全員	新しい緊急度と点数
チケットが完了した	自分の件が完了したとき	全員	全員	対応完了
チケットが待ちに戻った	自分の件が待ちに戻ったとき	全員	全員	離席などで担当が外れた
今の待ち一覧一式	送らない（自分の一覧は通常の取得）	つながった本人	つながった本人	接続した瞬間の待ち一覧と、担当者の空き
対応期限切れ	自分の件がオーバーしたとき	全員	全員	赤い警告
ダッシュボード更新	送らない	送らない	全員	数字のやり直し結果（30秒ごと）
デモの進み	送らない	送らなくてよい（待ち一覧の変化で足りる）	実行した管理者	何件まで作ったか

全員に送ってよいのは、待ち順の変化と「誰が引き取ったか」である。申請者には、他の顧客の待ち一覧・現場全体の数字・他人の本文を送らない。

接続の管理

項目	内容
実装内容	誰がつながっているかを管理する
内容	つながっている人、役割、つながった時刻
処理	接続・切断のたびに一覧を更新する。担当者が切れてから5分戻らなければ、担当を外す
完了条件	接続の状態が最新で、離席判定が動く

待ち順を Redis に置く理由

項目	内容
何をするか	対応待ちの「今の順番」を Redis に持たせる。点数が高いほど先
キー	<code>queue:tickets</code>
点数	ルール計算が出した点数
値	チケットのID
操作	作成・やり直しのときは並びへ載せる。 「次のタスクを引き取る」ときはいちばん点が高い1件を取り出して同時に消す。 完了や担当解除のときは並びから外す
どちらが正しいか	チケットの状態の正は PostgreSQL。Redis は順番を速く出すため。 起動時や食い違いを見つけたときは、データベースの対応待ちから順番を作り直す
完了条件	待ち一覧の順が点数のルールと一致する。 「次のタスクを引き取る」で、同時でも1件ずつ正しく渡る

PostgreSQL への保存

項目	内容
実装内容	チケット、顧客、ユーザー、対応の記録、操作の記録を残す
利用目的	障害のあとでも、チケットと対応期限の判定を復元する
完了条件	最新の状態がデータベースにある

再起動後の復旧

項目	内容
実装内容	サーバー再起動後、データベースの対応待ちから Redis の順番を作り直す
処理	対応待ちの点数をやり直して並び載せる
完了条件	再起動後も、引き取りと一覧が続けられる

プログラムの分け方

バックエンドは、内側（ルール）が外側（保存や通信）に依存しない形にする。

```

backend/
├─ domain/      ... チケット、プラン、点数の計算、担当者。保存先を知らない
├─ usecase/     ... 起票、次のタスクを引き取る、緊急度変更、完了、稼働更新、対応期限の確認、デモ
├─ infra/       ... PostgreSQL への保存、Redis の順番メモリ、画面への知らせ
└─ presentation/ ... HTTP の入口

```

通常のHTTPは次の順で処理する。

```

入口
↓
共通処理（認証など）
↓
画面・APIの受け口
↓
業務の手順
↓
保存 / 順番メモリ / 画面への知らせ

```

点数の計算ファイルは式だけを持つ。「Redis でやり直すか、データベースで計算するか」は知らない。順番メモリと、30秒ごとの処理がこの計算を呼ぶ。

認証・セキュリティ要件

認証方式、パスワード、Access Token、Refresh Token、HTTPS/WSS、CSRF、CORS、XSS、入力値の確認、エラー内容の隠し方は、NoteHub 要件定義書と同じ方針とする。本システムだけ違う点を以下に書く。

操作できる人かの確認

項目	内容
実装内容	チケット・顧客・ダッシュボードを触ってよい役割か確認する
API	毎回、役割を見る。申請者なら、そのチケットの本人かも見る
常時接続	つながるときに役割を見る。申請者へは他人の知らせを送らない
役割を変えたとき	その直後から、新しい役割でAPIと常時接続を制限する
完了条件	自分の役割以外の待ち一覧、現場全体の数字、他人のチケットを見たり操作したりできない

連続操作の制限

項目	内容
実装内容	短い時間に同じ操作が続いたら受け付けない
対象	会員登録、ログイン、Token再発行、起票、次のタスクを引き取る、緊急度変更、デモ開始
単位	ログイン前はIPアドレス、ログイン後はユーザー
方法	Redis で「残りの回数」を管理する
超えたとき	業務処理をせず、429 を返す

項目	内容
完了条件	上限以内は処理し、超えた操作は拒否する

入力値の確認

項目	内容
実装内容	API と常時接続で受け取った値を確認する
対象	必須かどうか、文字数、IDの形、緊急度・状態・プラン・種類・稼働状態の決められた値
タイトル	1～200文字
詳細・コメント	1～5000文字（終了コメントは2000文字まで）
不正なとき	データベースへ書かず、入力エラーを返す
完了条件	条件を満たさない入力は保存も配信もしない

データモデル

テーブル	内容	項目
User	アカウント	id, email, password_hash, name, role (applicant / agent / admin) , customer_id (空でも可) , status, created_at, updated_at
Customer	顧客	id, name, plan (free / pro / enterprise) , sla_minutes, created_at, updated_at
AgentState	担当者の稼働	user_id, status (available / busy / offline) , last_seen_at, updated_at
Ticket	問い合わせ	id, title, description, category, severity, status (open / in_progress / closed) , customer_id, created_by, assigned_agent_id (空でも可) , priority_score, sla_violated_at (空でも可) , sla_notified_at (空でも可) , created_at, claimed_at (空でも可) , closed_at (空でも可) , updated_at
TicketComment	対応・完了のコメント	id, ticket_id, user_id, body, kind (work_note / close_note / escalate_note) , created_at
TicketSeverityHistory	緊急度の変更履歴	id, ticket_id, from_severity, to_severity, reason, changed_by, created_at
RefreshSession	ログインの継続	id, user_id, token_hash, family_id, status, expires_at, created_at, rotated_at, revoked_at, last_used_at
LoginAttempt	ログイン失敗	id, email, ip_address, failed_count, locked_until, last_failed_at, created_at, updated_at
AuditLog	操作の記録	id, user_id, action, target_type, target_id, ip_address, details, created_at
APIRequestLog	APIの処理記録	id, request_id, user_id, method, path, status, latency_ms, error_code, created_at
WebSocketLog	常時接続の記録	id, user_id, connection_id, event_type, reason, ip_address, created_at
DemoRun	デモの実行	id, started_by, total_count, interval_sec, distribution, status, generated_count,

テーブル	内容	項目
		started_at, stopped_at

API

API	内容
POST /auth/signup	ユーザー登録（申請者）
POST /auth/login	ログイン
POST /auth/logout	ログアウト
POST /auth/refresh	AccessToken の再発行
GET /me	今ログイン中の人と役割、所属顧客
GET /customers	顧客一覧（管理者）
POST /customers	顧客作成（管理者）
PATCH /customers/:id	顧客更新（管理者）
GET /users	ユーザー一覧（管理者）
PATCH /users/:id	役割・所属顧客・状態の更新（管理者）
POST /tickets	チケット起票（申請者）
GET /tickets	自分のチケット一覧（申請者）
GET /tickets/:id	チケット詳細（見られる人だけ）
GET /queue	待ち一覧（担当者・管理者）
POST /queue/claim	次のチケットを引き取る（担当者）
POST /tickets/:id/escalate	緊急度の変更
POST /tickets/:id/comments	対応メモの追加
POST /tickets/:id/close	対応完了
POST /tickets/:id/release	管理者が担当を外し、待ち一覧へ戻す
PATCH /agent/status	担当者の稼働状態
GET /admin/dashboard	現場の数字（管理者）
POST /admin/demo/start	デモ開始（管理者）
POST /admin/demo/stop	デモ停止（管理者）
GET /health	稼働確認
WS /ws	役割ごとの即時更新

AccessToken

項目	内容
形式	JWT
有効期限	15分
保持場所	ブラウザのメモリ上のみ（localStorage / sessionStorage には置かない）
入れる情報	user_id、役割、発行日時、有効期限
用途	REST API と常時接続の本人確認

セキュリティ・運用要件

連続操作の制限

項目	内容
やり方	残回数をバケツに見立てて減らす方式
実装	Redis と、途中で切れない一連のスクリプト
超えたとき	429 Too Many Requests

操作	数える単位	上限の目安
ログイン	IPアドレス+アカウント	5回分（1分あたり1回復）
会員登録	IPアドレス	3回分（1時間あたり）
Token再発行	ユーザー	10回分（1分あたり5回復）
起票	ユーザー	10回分（1分あたり5回復）
次のタスクを引き取る	ユーザー	10回分（1分あたり5回復）
緊急度変更	ユーザー	10回分（1分あたり5回復）
デモ開始	ユーザー	3回分（10分あたり）

ログイン失敗の制限

項目	内容
目的	パスワードの総当たりを防ぐ
上限	5回失敗で一時的にログイン不可
ロック	開発15分、本番想定1時間
記録	アカウント単位とIPアドレス単位

通信の安全

項目	内容
API	本番は HTTPS 必須
常時接続	本番は暗号化必須
Cookie	Secure、HttpOnly、SameSite
CORS	フロントエンドのドメインだけ許可

再ログイン用（Refresh Token）

項目	内容
保存場所	JavaScript から読めない Cookie
データベース	ハッシュだけ保存する
有効期限	30分（本番想定14日）
使い直し	使うたびに新しい印を出し、古い印は無効にする
無効な印の再利用	強制ログアウトし、記録に残す
CSRF	Cookie を使うAPIは、Cookie とヘッダーの値を照合する

ログ

種別	記録するもの
APIの処理	request_id、method、path、status、かかった時間、user_id、error_code
操作の記録	ログイン、ログアウト、無効な印の再利用、ログイン失敗制限、起票、引き取り、緊急度変更、完了、担当解除、顧客変更、役割変更、デモ開始・停止
制限と接続	回数制限超え、常時接続の強制切断

インフラ・デプロイ

項目	内容
フロントエンド	React、TypeScript、Vite、Tailwind CSS。Docker コンテナで出す
バックエンド	Go (Echo) の REST と常時接続。Docker コンテナ
データの本体	PostgreSQL (GORM)
Redis (待ち順)	点数順の並びと、連続操作の回数
Redis (配信)	サーバーが複数あるときの画面への知らせ
定期処理	点数のやり直しと、対応期限切れの検知を30秒間隔
コンテナ	Docker Compose でフロント、バック、PostgreSQL、Redis をまとめて起動
CI/CD	GitHub Actions でビルド、テスト、デプロイ

テスト要件

バックエンド

対象	主な確認	完了条件
パスワード	ハッシュ化、正しい照合、違うパスワードの拒否	データベースとログに平文が残らない
JWT	発行、署名、期限切れ、改ざん	有効な Access Token の利用者だけが認証必須 API を使える
会員登録	メール、表示名、パスワード。最初の役割は申請者	正しい入力で申請者アカウントができる
ログイン	成功、失敗、停止中、存在しないメール	失敗理由を区別しない。5回失敗で一時的にログインできない
再発行とログアウト	Cookie、CSRF、使用済みの印の無効化、再利用の検知	古い印では再発行できない。再利用時はそのログインをすべて無効にする
申請者と顧客	所属は1つの社外顧客。Free / Pro / Enterprise	所属が無い起票は拒否する。プランは所属顧客から付き、手入力しない
起票	件名、詳細、緊急度、種類。対応待ちで保存	チケットができ、点数順の並びに入る。担当者と管理者は起票できない
申請者の一覧と詳細	自分の件だけ。更新が新しい順	他人の件は一覧に出ない。他人の詳細は「見つからない」
点数	判定表の全行。待ち時間の加点は99分まで	緊急度が違うときは高い方が先。期限切れの Free は、時間内の Enterprise より先。同じ緊急度で時間内ならプランが高い方が先。どちらも期限切れならプランが高い方が先。条件が同じなら待ちが長い方が先。Enterprise の質問より、Free の全停止が先
待ち一覧	点数が高い順。同じ点なら先に作られた順。同時なら ID が小さい方	申請者は拒否する。担当者と管理者は点数の順で見られる
次を引き取る	待機中だけ。対応中は1件まで。同時押し	同じチケットは一人にしか付かない。作業中と離席は拒否する
緊急度	担当者は自分の対応中だけ上げられる。下げは管理者だけ	対応待ちなら並びの位置が変わる。完了した件は変えられない
対応完了	終了コメント必須。担当者本人または管理者	完了になり、待ち一覧から消える。担当者は待機中に戻る
待ちに戻す	離席、管理者の担当外し、切断から5分	担当者を引き取った時刻は空。作った時刻は変わらない。順番メモリに載る
対応期限	30秒ごとのやり直し。初めて担当が付くまで	過ぎた件は加点され、知らせは1回。担当が付いたら赤は消える
ダッシュボード	待ち件数、対応中、平均時間、期限切れ、担当者の空き	申請者と担当者は拒否する。管理者の数字が定義どおり
デモ	件数、間隔、均等または障害多め。通常の起票と同じ道	作った件が待ち一覧に載る。実行中の再開は拒否する

対象	主な確認	完了条件
回数制限	会員登録、ログイン、再発行、起票、引き取り、緊急度変更、デモ開始	上限以内は処理する。超えたら 429 で、業務処理はしない
入力	件名 1~200、詳細 1~5000、終了コメント 1~2000、理由 1~500	条件を満たさない入力は保存も配信もしない
WebSocket	接続時の Access Token。役割ごとの知らせ	申請者には自分のチケットだけ。待ち一覧と現場の数字は送らない
順番メモリ	キー <code>queue:tickets</code> 。値はチケット ID	再起動後、対応待ちから順番を作り直せる

フロントエンド

Vitest で次を確認します。

- ログインと会員登録。未ログインで必要な画面を開くとログイン画面へ移る。
- ログイン中にログイン画面を開くと、役割ごとのホームへ移る。申請者は `/tickets`、担当者は `/agent/queue`、管理者は `/admin/dashboard`。
- 申請者は自分のチケット一覧、新しい問い合わせ、自分の詳細だけを開ける。
- 申請者の起票画面にプランの入力が無い。
- 担当者は待ち一覧を見られ、待機中だけ「次を引き取る」が押せる。
- 対応期限を過ぎた行は赤く表示される。
- 管理者はダッシュボード、全件、顧客、利用者、デモを開ける。顧客登録は Free / Pro / Enterprise だけ。
- 自分の役割以外の画面は開かない。
- 件名、詳細、コメントは文字として表示し、入力した HTML をそのまま実行しない。
- API のエラーは、内部情報を出さない文面に表示する。